



COS 132 – Imperative Programming

Foundation Test 1 Memo

Worked Solutions

Module:	COS 132 – Imperative Programming
Assessment:	Foundations Test 1 (ST2 Programme)
Set by:	Swift Tuition
Total Marks:	22

Programme note. This material is for Day 1 of the 7-day Swift Tuition COS 132 programme. The premium course is available at www.swifttuition.com/tuks/cos132. Learners watching before ST2 2026 may use coupon code **200COS132** at checkout for R200 off the full price. All three foundation tests are included; one test can be taken now, and the remaining tests can be taken anytime after ST2. Where units are referenced, they are Swift Tuition learning units based on how the module's lecture content was split, covering lectures 1–16.

1 Overview

Good day, good people. Below is the worked solution for Foundations Test #1. Read the reasoning, not just the answers — the point of looking at a memo is to make the next question of this kind feel obvious instead of tricky.

Total marks: 22

2 Question 1 [8 marks]

(a) True or False

i. **False.** [1 mark]

`cin` looks like it should read whatever you type, but it actually stops the moment it hits any whitespace. The way to picture this: imagine a paper tray sitting between your keyboard and your program. Every time you type a line and press Enter, that whole line drops onto the tray. Your program never sees the keyboard directly — it only takes from the tray.

`cin >> name` is a hand that reaches into the tray, grabs characters until it hits a space (or tab, or newline), and pulls just those characters into `name`. Anything past the first space **stays on the tray**.

So if a user types John Smith and presses Enter, the tray now holds John Smith\n (the \n is the newline from Enter). `cin >> name` grabs John and leaves Smith\n sitting there. If the next line of your code is `cin >> surname`, the hand reaches in again — it does not prompt the user — and grabs Smith from what's already on the tray. The user types nothing in between, because the tray wasn't empty.

This is exactly why typing a name with a space breaks programs that use `cin >>` for the whole name. To grab a whole line including spaces, you use `getline(cin, name)` instead. `getline` reaches into the tray and takes everything up to the next newline.

ii. **True.** [1 mark]

`setw` lives in the `<iomanip>` header. So does `setprecision` and the `fixed` / `scientific` manipulators. Whenever you're formatting numeric output (column widths, decimal places), you'll need that header.

iii. **True.** [1 mark]

This is the most common source of "my if-statement isn't working" bugs. `=` is **assignment** (put this value into the variable on the left). `==` is **comparison** (check whether two things are equal). Mixing them up is so common that most compilers will warn you when you write `if (x = 5)` because they suspect you meant `==`.

iv. **False.** [1 mark]

The rule is the opposite: default values must be on the **rightmost** parameters. The reason is positional. When you call `f(1, 2)`, the compiler matches 1 to the first parameter and 2 to the second. If you allowed defaults on the left, the compiler couldn't tell which argument went where.

So this is legal:

```
int compute(int a, int b = 5, int c = 10);
```

But this is a compile error:

```
int compute(int a = 1, int b, int c = 10); // ERROR
```

Once a parameter has a default, every parameter to its right also needs one.

(b) Multiple choice

i. **B.** `const double PI = 3.14159;` [1 mark]

Two things make a constant work in C++: a meaningful name, and a fixed value. `const double` does both — `double` because pi has a fractional part, `const` so the compiler protects you from accidentally

reassigning it later.

Why the others don't work:

- **A** stores `pi` as `int`, which truncates the fractional part. Your "pi" would be 3. Useless.
- **C** declares a `const double` but never initialises it. C++ requires every `const` to be given a value at the moment you declare it (otherwise it could never have one, since `const` means "can't change later").
- **D** is the old C-style preprocessor syntax, and even then it's written incorrectly. The textbook way for COS 132 is `const`, not `#define`.

ii. A. `15 / 2` [1 mark]

This one catches people. `15 / 2` looks like it should give 7.5, but both 15 and 2 are `int` literals. When both operands of `/` are `int`, C++ does **integer division** — the fractional part is thrown away. So `15 / 2 = 7` exactly.

The other options:

- **B** gives 1 (modulus is the remainder after division: $15 = 7 \times 2 + 1$).
- **C** and **D** both have a `double` literal in them (`15.0` or `2.0`), which forces floating-point division. They both give 7.5, not 7.

The takeaway: pay attention to the *types* of the operands, not just the values. As soon as one operand becomes a `double`, the whole expression promotes to `double`.

iii. B. `<iostream>` [1 mark]

`<iostream>` is the input/output stream header. It's where `cin`, `cout`, `cerr`, `endl` and friends live. Almost every COS 132 program will start with `#include <iostream>`.

The other options:

- **A** `<stdio.h>` is the C-style I/O header (`printf`, `scanf`). Valid C++ but not what COS 132 teaches.
- **C** `<string>` gives you the `string` type, but not `cin` and `cout`.
- **D** `<iomanip>` gives you the formatting manipulators (`setw`, `setprecision`) but not `cin`/`cout` themselves.

iv. A. `int add(int, int);` [1 mark]

A function declaration (also called a prototype) tells the compiler that a function exists, what it returns, and what it takes. The minimum form needs the return type, the function name, and the parameter types. Names for the parameters are optional in a declaration.

Why the others fail:

- **B** has return type `void`, which means the function returns nothing. The question asked for a function that *returns the sum*, so `void` is wrong.
- **C** uses Pascal-style return-type-after-colons syntax, which isn't C++.
- **D** prefixes with the keyword `function`, which doesn't exist in C++. (You're thinking of JavaScript or Python.)

3 Question 2 [6 marks]

(a) Output: 2 1 [2 marks]

Both `a` and `b` are declared as `int`. When you divide two `int` values with `/`, you get integer division — the fractional part is discarded. `7 / 3 = 2` (the actual answer is 2.33, but the .33 is thrown away).

`a % b` is the modulus operator. It gives you the remainder after dividing. `7 ÷ 3` is 2 remainder 1, so `7 % 3` is 1.

The two values are printed with one space between them.

Watch out: if you wrote `2.33 1`, you assumed C++ would automatically promote to floating-point. It won't, unless one of the operands is itself a floating-point value. Both `ints` on either side of `/` keeps you in integer-division land.

(b) Output: 7 5 7 [2 marks]

The trick here is the difference between `x++` (post-increment) and `++x` (pre-increment).

Trace it line by line, starting with `x = 5`:

1. `int y = x++;`
 - `x++` is **post**-increment. It evaluates to the **current** value of `x` first, then increments `x`.
 - So `y` takes the value 5, and **after** that, `x` becomes 6.
2. `int z = ++x;`
 - `++x` is **pre**-increment. It increments `x` first, then evaluates to the new value.
 - So `x` becomes 7, and `z` takes the new value 7.
3. Print `x, y, z` → 7 5 7.

The mental model: post-increment uses the value first and bumps later. Pre-increment bumps first and uses the new value. The `++` placement tells you which.

If you got 7 6 7, you mixed up which one captures the pre and which captures the post. That's worth 1 of 2 marks if your reasoning was clearly traced.

(c) Output: 8 4.5 [2 marks]

This question is testing **function overloading**. The way to picture it: imagine you and your sibling both share the name `compute` because you do similar jobs in the family business. When someone shouts `compute(...)`, the family decides who answers based on what was passed in. You handle integer requests; your sibling handles fractional requests. Same name, different specialty.

In C++, when the compiler sees a call like `compute(4)`, it looks at every function named `compute` and asks: which one of these can take a single `int`? It picks the matching one, generates the call to that specific function, and that's what runs.

Now resolve the two calls:

`compute(4)` — the 4 is an `int` literal. The `int compute(int)` overload matches. It returns `4 * 2 = 8`.

`compute(4.0)` — the 4.0 is a `double` literal (the .0 is what makes it floating-point). The `double compute(double)` overload matches. It returns `4.0 + 0.5 = 4.5`.

Two important rules to keep in mind:

1. **The compiler picks the overload based on what you pass, not what you ask back for.** Even though one overload returns `int` and the other returns `double`, the *return type* doesn't disambiguate. If C++ allowed return-type-based picking, the compiler couldn't tell them apart from a call site like `compute(4)` where you don't say what you want back. So the language was designed to ignore return type for overload resolution.
2. **An exact-match wins over a conversion.** If you call `compute(4)` and there's both an `int` and a `double` overload, C++ picks `int` because the argument is already an `int` — no conversion needed. It would only fall back to converting 4 into 4.0 and using the `double` overload if the `int` version didn't exist.

Watch out: a common mistake is reading 4.0 as just 4 and picking the `int` overload. The `.0` matters — it's what tells the compiler "this is a `double`". Without that single character, you'd be calling the `int` version.

4 Question 3 [4 marks]

Expected answer

```
void printLine(int n) {  
    for (int i = 0; i < n; i++) {  
        cout << "*" << endl;  
    }  
}
```

How to think about it

The Logo procedure does three things:

1. Takes one parameter (:N).
2. Repeats :N times.
3. Each time, prints a * followed by a newline.

Each of those maps cleanly onto a C++ idea:

1. :N becomes a function parameter `int n`.
2. REPEAT :N [...] becomes a `for` loop that runs `n` times.
3. PRINT "*" becomes `cout << "*" << endl`; (or any equivalent way of writing a star and a newline).

Since the Logo procedure doesn't return anything, the C++ function returns `void`.

Marks breakdown

- 1 mark: function signature `void printLine(int n)`
- 1 mark: loop runs the correct number of times (the body executes `n` times)
- 1 mark: body prints a star followed by a newline
- 1 mark: overall correctness — no off-by-one, no extra newlines, compiles cleanly

Other valid answers

Using `while`:

```
void printLine(int n) {  
    int i = 0;  
    while (i < n) {  
        cout << "*" << endl;  
        i++;  
    }  
}
```

Same output, full marks.

Using `\n` instead of `endl`:

```
cout << "*\n";
```

Same effect (a star followed by a newline). Full marks.

Using a char literal '*' instead of the string literal "*": equivalent. Full marks.

Using `i = 1; i <= n` instead of `i = 0; i < n`: the loop still runs `n` times. Full marks.

Common pitfalls

- **Forgetting the newline.** `cout << "*";` without `endl` puts every star on the same line. The Logo source clearly produces one star per line.
- **Off-by-one.** `i <= n` with `i` starting at 0 runs the loop `n+1` times — one too many. Either start at 0 and use `<`, or start at 1 and use `<=`. Pick one and stick with it.
- **Returning `int` instead of `void`.** The Logo procedure produces no value, so the C++ version shouldn't either.

5 Question 4 [4 marks]

(a) Declare an integer variable called `count` and initialise it to zero. [1 mark]

```
int count = 0;
```

The `= 0` is the **initialiser**. Without it, the variable holds whatever garbage was already in that memory location, and your first read of `count` could give you anything. Always initialise before reading.

A second valid form is `int count(0);` — the constructor-style initialisation. Same effect, also full marks.

(b) Read a full line of text into `name`. [1 mark]

```
getline(cin, name);
```

This is the same idea as Q1(a)(i). `cin >> name` would only read up to the first whitespace, and the question specifically says "including spaces". `getline` reads up to the next newline, so spaces are kept.

A bare `cin >> name` does **not** earn the mark — it fails the very thing the question is asking about.

(c) Print `total` with exactly 2 digits after the decimal point. [1 mark]

```
cout << fixed << setprecision(2) << total;
```

You need **two** manipulators working together:

- `fixed` tells `cout` to use fixed-point notation (so 123.45 doesn't get printed as 1.2345e2).
- `setprecision(2)` tells `cout` how many digits to show.

If you only use `setprecision(2)`, you get total significant digits across the whole number, not digits after the decimal. So 1234.5 prints as 1.2e+03, which is not what you want. The `fixed` switches the meaning of `setprecision` to "decimals after the point".

Both manipulators live in `<iomanip>`, so that header would also need to be included (the question said you can assume necessary headers are there).

Watch out: `cout << total;` alone gives default formatting, which doesn't guarantee exactly 2 decimals. Zero marks for that.

(d) Calculate the average of `a` and `b` as a `double`. [1 mark]

```
double avg = static_cast<double>(a + b) / 2;
```

Here's why the cast matters. If you wrote `(a + b) / 2` without the cast, both operands are `int` (because `a` and `b` are `int`, and 2 is an `int` literal). That's integer division. So if `a + b` is 7, the result is 3, not 3.5. The cast forces the *left* side of the division to be `double`. Once one side of `/` is `double`, the whole division is floating-point, and `7 / 2` becomes 3.5.

A second valid form is:


```
double avg = (a + b) / 2.0;
```

The `.0` on the `2` makes it a `double` literal, so the division is floating-point for the same reason. Either is full marks. Both are common in past papers.

Watch out: `double avg = (a + b) / 2;` is **not** correct. The right-hand side does integer division first (because both operands are `int`), then assigns the truncated integer result to a `double`. The `.5` is lost before it ever reaches `avg`.

6 After this test

This was the diagnostic baseline. Roughly:

- **18–22 marks:** your foundation is solid. Move on to Unit G with confidence.
- **13–17 marks:** mostly there. Identify which questions caught you and revisit those specific topics in Units C–F.
- **8–12 marks:** the foundation has real gaps. Don't push into ST2 work yet — go back to Units C–F and rebuild before this programme is useful to you.
- **Below 8:** start with the unit videos for Units C and D first.

Logo coverage

This test contained one Logo→C++ conversion (Q3) — a single `REPEAT` loop converting to a single `for` loop. Per the COS 132 lecturer's confirmation that ST2 can ask Logo→C++ conversions, this is the easy end of what you might see.